Unit 7: Number Theoretic Algorithms

Concept of Number Theoretic Notation

- Elementary Number theoretic notation is based on the set of integers Z ={… … , -2,-1,0,1,2,… …} and the set of natural numbers N={ 1,2,3,4,… …}
- Divisibility and divisor: In number theory, the notation d|a i.e. d divides a means a =kd for some integer k. Here a is multiples of d and d is a factor of a.
- We can write k|0 for any integer k >0 i.e. 0 is divisible by every integer greater than 0.
- d|a iff - d|a i.e. no generality is lost by defining the divisors to be non-negative or negative.
- A divisor of ‘a’ is not greater than ‘a’ and at least 1.
- For example , Divisor of 16 is 1,2,4,8,16

Composite and Prime Numbers

- Composite Numbers have a divisor other than itself and one.
  - For example 4|20 means that 20 = 5 * 4 i.e. 4 is a factor of 20.
- The divisors of 12 are 1,2,3,4,6 and 12
- Prime numbers have no divisors but 1 and itself
- First 10 Primes
- 2, 3, 5, 7, 11, 13, 17, 19, 23, 29

Common Divisors and GCD

- If h|m and h|n then h is called a common divisor
- A common divisor is a number that is a factor of both numbers
- The greatest common divisor is the largest factor for both numbers
- This is denoted gcd (n,m)
- For example gcd(12,15) = 3
- For any two integers n and m where m != 0 the quotient is given by
- q = floor(n/m) e.g. 15/4= 3
- The remainder r of dividing n by m is given by
- r = n − qm e.g. r= 15-3*4 = 3

Greatest Common Divisor

- Let **n** and **m** be integers, not both 0 and let
- ***d*** = min { ***in + jm*** such that ***i,j*** $\in$ ***Z*** and ***in + jm > 0***} , ***Z*** a set of integers
- That is, ***d*** is the smallest positive linear combination of ***n*** and ***m***
- For example we know gcd(12, 8) = 4,
- the smallest linear combination is
- ***4 = (3*12) + (−4)*8***
- Now suppose we have n ≥ 0 and m > 0 and r = n mod(m) then
- ***gcd(n , m) = gcd(m , r)***
- so ***gcd(64 , 24) = gcd(24, 16)***

  = gcd(16, 8)

  = gcd(8, 0)

  = 8

Prime Factorization

- Two integers are relatively prime because the gcd of them is 1
- For example ***gcd(12, 25) = 1*** so they are relatively prime
- If ***h*** and ***m*** are relatively prime and ***h*** divides ***nm***, then ***h*** divides ***m***.
- That is ***gcd(h,m) = 1*** and ***h|nm*** implies ***h|n***
- e.g.: ***gcd(10,13)=1, 5|10*13 => 5|10***

Least Common Multiple

- For n and m where they are both nonzero, the least common multiple is denoted ***LCM(n,m)***
- For example ***LCM(6,9) = 18*** because ***6|18*** and ***9 |18***
- The ***LCM(n,m)*** is a product of primes that are common to m and n, where the power of each prime in the product is the larger of its orders in n and m
- So ***12 = 2*2*3*1 and 45 = 3*3*5*1***
- ***so LCM(12,45) = 2*2*3*3*5*1 = 180***

Euclid's Algorithm

Euclid's Algorithm gives us a straight forward way to find the GCD, the largest number that divides both

of two numbers

```
int GCD(int n, int m)
{
    If(m == 0)
        return n;
    else
        return GCD(m, n mod m);
}
```

Analysis: GCD is resursive algorithm. At every recursion, problem is divided into two parts one is b and another is a mod b. So size of problem becomes n/2.

Dividing and merging it takes constant time , So recurrence relation for this algorithm is

$T(n) = T(n/2) + O(1)$

Solving his recurrence we get : $T(n) = O(\log n)$ *i.e.* $O(\log(\max(m,n)))$

Extension to Euclid's Algorithm

- The extended Euclid's algorithm *also finds integer coefficients x and y such that:* ***ax + by = gcd(a, b)*** *where the x and y are known as Bezout's identity coefficients and x and y may be zero or negative.*

- *The algorithm takes as input a pair of nonnegative integers and returns a tripe of the form* ***(d,x,y)*** *that satisfies the given equation.*
- *It is used for finding the GCD of two positive integers 'a' and 'b' and writing this GCD as an integer linear combination of a and b.*

Examples:

- **Input:** a = 30, b = 20 **Output:** gcd = 10, x = 1, y = -1
  (Note that 30*1 + 20*(-1) = 10)

- **Input:** a = 35, b = 15 **Output:** gcd = 5, x = 1, y = -2
- (Note that 35*1 + 15*(-2) = 5)

Extension to Euclid's Algorithm

- The extended Euclidean algorithm updates the results of gcd(a, b) using the results calculated by the recursive call gcd(b%a, a).
- Let values of x and y calculated by the recursive call be x_1 and y_1. x and y are updated using the below expressions.

$ax + by = gcd(a, b)$

$gcd(a, b) = gcd(b\%a, a)$

$gcd(b\%a, a) = (b\%a)x_1 + ay_1$

$ax + by = (b\%a)x_1 + ay_1$

$ax + by = (b - [b/a] * a)x_1 + ay_1$

$ax + by = bx_1 - [b/a] * ax_1 + ay_1$

$ax + by = a(y_1 - [b/a] * x_1) + bx_1$

Comparing LHS and RHS,

$x = y_1 - [b/a] * x_1$

$y = x_1$

Extension to Euclid's Algorithm

```
int gcdExtended(int a, int b, int *x, int *y)
{
    if (a == 0) // Base Case
    {
        *x = 0;
        *y = 1;
        return b;
    }
  int x1, y1; // To store results of recursive call
 int gcd = gcdExtended(b%a, a, &x1, &y1);
  // Update x and y using results of recursive call
 *x = y1 - (b/a) * x1;
 *y = x1;
  return gcd;
}
```

How does Extended Algorithm Work?

- As seen above, x and y are results for inputs a and b,

  $a.x + b.y = gcd$ ----------(1)

- And x_1 and y_1 are results for inputs b%a and a

  $(b\%a).x_1 + a.y_1 = gcd$

- When we put $b\%a = (b - (\lfloor b/a \rfloor).a)$ in above, we get following. Note that $\lfloor b/a \rfloor$ is floor(b/a)

  $(b - (\lfloor b/a \rfloor).a).x_1 + a.y_1 = gcd$

- Above equation can also be written as below

  $b.x_1 + a.(y_1 - (\lfloor b/a \rfloor).x_1) = gcd$ ----------(2)

- After comparing coefficients of 'a' and 'b' in (1) and (2), we get following,

  $x = y_1 - \lfloor b/a \rfloor * x_1$

  $y = x_1$

Complexity of Extended Euclid’s algorithm

- The extended euclidean algorithm takes the same time complexity as Euclid's GCD algorithm as the process is same with the difference that extra data is processed in each step.
- So Time and Space complexity will by ***O(log(max(a,b))***
- **Usefulness of Extended Euclidean Algorithm**

  The extended Euclidean algorithm is particularly useful when a and b are relatively prime (or gcd is 1). Since x is the modular multiplicative inverse of “a modulo b”, and y is the modular multiplicative inverse of “b modulo a”.
- **Applications of the above algorithms**

  The computation of the modular multiplicative inverse is an essential step in RSA public-key encryption method. There's where the Extended Euclidean Algorithm comes into play.

Chinese Remainder Theorem

- Around A.D. 100, the Chinese mathematician solved the problem of finding those integers x that leave remainders 2, 3, and 2 when divided by 3, 5, and 7 respectively. One such solution is $x = 23$; all solutions are of the form $23 + 105k$ for arbitrary integers k.
- The "Chinese remainder theorem" provides a correspondence between a system of equations modulo a set of pairwise relatively prime moduli (for example, 3, 5, and 7) and an equation modulo their product (for example, 105).
- The *Chinese Remainder Theorem* states that if one knows the remainders of the Euclidean division of an integer n by several integers, then one can determine uniquely the remainder of the division of n by the product of there integers, under the condition that divisors are pairwise co-prime

Chinese Remainder Theorem

- We are given two arrays num[0..k-1] and rem[0..k-1]. In num[0..k-1], every pair is coprime (gcd for every pair is 1).

- Let num[0], num[1], …num[k-1] be positive integers that are pairwise coprime. Then, for any given sequence of integers rem[0], rem[1], … rem[k-1], there exists an integer x solving the following system of simultaneous congruences.

  x % num[0] = rem[0],
  x % num[1] = rem[1],
  .....
  x % num[k-1] = rem[k-1]

- We need to find minimum positive number x from above equations.

  – Basically, we are given k numbers which are pairwise coprime, and given remainders of these numbers when an unknown number x is divided by them. We need to find the minimum possible value of x that produces given remainders.

Chinese Remainder Theorem

Examples :

- Input: num[] = {5, 7}, rem[] = {1, 3}
- Output: 31
- Explanation: 31 is the smallest number such that:
  (1) When we divide it by 5, we get remainder 1.
  (2) When we divide it by 7, we get remainder 3.

- Input: num[] = {3, 4, 5}, rem[] = {2, 3, 1}
- Output: 11

- Explanation: 11 is the smallest number such that:
  (1) When we divide it by 3, we get remainder 2.
  (2) When we divide it by 4, we get remainder 3.
  (3) When we divide it by 5, we get remainder 1.

Solving Modular Linear Equations

- A congruence of the form ax≡b(mod n) where x is an unknown integer is called a linear congruence in one variable.
- We now consider the problem of finding solutions to the equation $ax \equiv b \pmod{n}$, where a>0 and n>0.

We assume that a,b,n are given so that we have to fine all values of x modulo n, that satisfy given equation. This lead to zero, one or more than one solution.

- First we have to compute the gcd(a,n) let it is d.
- If the gcd(a,n) divides b, then there are d congruent solution to the given modular linear equations and there are d number of solutions.
- If d does not divides b, then there is no congruent solution to the equation.

Solving Modular Linear Equations (Example)

- Solve $5x \equiv 2 \pmod{26}$
  - Solution: Here, a=5, b=2 and n=26
  - d= gcd(5,26) =1 and 1|2 i.e. d|b so there is exactly 1 incongruent solution modulo 26.
  - By Euclidean algorithm, we have (see on box)

    26 = 5 *5+1

    5 = 1*5+0

    Now, 1=26-5*5

    5*5 =26-1

    $5*5 \equiv 26-1 \pmod{26}$

    $5(5) \equiv 0-1 \pmod{26}$

    $5(5) \equiv -1 \pmod{26}$

    $5(-5) \equiv 1 \pmod{26}$

    $5(-10) \equiv 2 \pmod{26}$

```
5)26(5
-25
-------
1)5(5
-5
--------
0
```

Solving Modular Linear Equations

- We now consider the problem of finding solutions to the equation $ax \equiv b \pmod{n}$, where a>0 and n>0.

We assume that a,b,n are given so that we have to find all values of x modulo n, that satisfy given equation. This lead to zero, one or more than one solution.

Algorithm:

MODULAR-LINEAR-EQUATION-SOLVER(*a,b,n*)

1 (*d,x',y'*)= *EXTENDED-EUCLID(*a,n*)*

2 if *d* | *b*

3 then $x_0 = x'(b/d) \bmod n$

4 for *i*= 0 to *d* - 1

5 do print $(x_0 + i(n/d)) \bmod n$

6 else print "no solutions“

– The running time of MODULAR-LINEAR-EQUATION-SOLVER is $O(\log n + \gcd(a,n))$ arithmetic operations, since EXTENDED-EUCLID takes $O(\log n)$ arithmetic operations, and each iteration of the **for** loop of lines 4-5 takes a constant number of arithmetic operations.

Solving Modular Linear Equations

- As example of the operation of this procedure:
  - consider the equation $14x \equiv 30 \pmod{100}$ (here, $a = 14$, $b = 30$, and $n = 100$).
  - Calling EXTENDED-EUCLID in line 1, we obtain $(d, x, y) = (2,-7,1)$.
  - Since $2 \mid 30$, lines 3-5 are executed.
  - In line 3, we compute $x_0 = (-7)(15) \bmod 100 = -5$. The loop on lines 4-5 prints the two solutions: -5 and 45. i.e. *(-5+1(100/2))*

```
#include<iostream>
using namespace std;
int gcdExtended(int a, int b, int *x, int *y)
{
    // Base Case
    if (a == 0)
    {
        *x = 0;
        *y = 1;
        return b;
    }
     int x1, y1; // To store results of recursive call
    int gcd = gcdExtended(b%a, a, &x1, &y1);
     // Update x and y using results of
    // recursive call
    *x = y1 - (b/a) * x1;
    *y = x1;

    return gcd;
}
```

```
void MODULAR_LINEAR_EQUATION_SOLVER(int a,int b,int n)
{
      int d,x,y;
      d=gcdExtended(a, n, &x, &y);
      cout<<"Output of ExtEculit: (d,x,y)= ("<<d<<"," <<x
      <<","<<y<<")"<<endl;
      cout<<"Solution of equations:";     if(b%d==0)
      {   int x0 = x*(b/d)%n;
                for(int i= 0;i<d;i++)
                        cout<<(x0 + i*(n/d))%n<<endl;
 }
      else cout<<"no solutions";
}

int main()
{
    int a = 14, b = 30,n=100;    // eq: 14x=30mod100
    MODULAR_LINEAR_EQUATION_SOLVER(a,b,n);
    return 0;
}
```

Output:

- Output of ExtEculit:(d,x,y)=(2,-7,1)
- Solution of equations:-5 45